
RP11: Mixed-Integer Hyperparameter Optimization using CMA-ES

1 Description

The Covariance Matrix Adaptation Evolution Strategy (CMA-ES) [1] is widely recognized as a state-of-the-art optimizer for continuous black-box functions, demonstrating high performance, particularly on problems that are ill-conditioned, non-separable, or multimodal [2]. CMA-ES generates candidate solutions from a multivariate normal distribution and adaptively updates its parameters (mean vector m , step-size σ , and covariance matrix C) based on the ranking of evaluated solutions [3].

CMA-ES has been successfully applied to numerous machine learning tasks, including direct policy search in Reinforcement Learning and, critically, Hyperparameter Optimization (HPO) of Deep Neural Networks [4].

HPO problems frequently involve a mix of continuous parameters (e.g., learning rates, decay factors) and discrete or integer parameters (e.g., number of layers, batch size). While standard CMA-ES is designed for continuous variables, a naive approach to handling integer variables (e.g., discretization by rounding) can lead to premature convergence to non-optimal points, resulting in search stagnation [5].

To address this challenge, the CMA-ES with Margin (CMAwM) method was proposed as an extension of CMA-ES tailored for discrete and mixed-variable optimization problems on sets of points [5, 6]. CMAwM employs margin correction to modify the covariance matrix, ensuring that the marginal generation probability of neighboring discrete points remains above a predefined target threshold (margin). This prevents the search distribution from collapsing onto a single non-optimal integer value prematurely.

2 Objectives

The goal of this exercise is to apply the CMA-ES algorithm (the original version, CMAwM, or any other variant), to optimize the hyperparameters of a neural network model designed for a benchmark classification task (e.g., MNIST).

The project should:

- **Setup the Benchmark:** Define a black-box objective function $f(\mathbf{x})$, where $\mathbf{x}$ is a mixed-integer vector of hyperparameters, and the output $f(\mathbf{x})$ is the validation error (or loss) of the trained model, which should be minimized.
- **Consider CMAwM:** Consider the possibility of implement CMA-ESwM, or utilize a specialized Python library to deal with the mixed-variable problem.
- **Compare Performance:** Compare the performance of the implemented variant against the results obtained using a baseline CMA-ES approach that implements naive discretization (e.g., rounding continuous samples to the nearest integer).

3 Suggestions

1. **Tooling:** Consider using the Python library such as `cmaes`¹ [7], `pycma`², or `nevergrad`³ to handle the mixed continuous and integer hyperparameter space efficiently
2. **Hyperparameters:** Select a set of hyperparameters to tune, including both continuous and integer dimensions, such as:
 - Continuous: Learning rate ($\alpha \in [10^{-5}, 10^{-1}]$), weight decay ($\lambda \in [0, 0.1]$).
 - Integer: Number of convolutional layers ($L \in \{1, 2, 3, 4\}$), filter size (e.g., $K \in \{3, 5, 7\}$).
3. **Evaluation:** To assess performance robustly, use standard metrics over multiple trials, typically considering several target values for the objective function. Alternatively, use the benchmark definitions from the BBOB suite [8, 2], although these are typically continuous.
4. **CMA-ES Foundations:** For a deeper understanding, consult tutorials regarding the core CMA-ES components: the overall algorithm structure, the role of the mean vector and covariance matrix adaptation, and step-size control via the evolution path [9].

References

- [1] N. Hansen and A. Ostermeier. Adapting arbitrary normal mutation distributions in evolution strategies: The covariance matrix adaptation. In *Proceedings of the 1996 IEEE International Conference on Evolutionary Computation*, pages 312–317, 1996.
- [2] Nikolaus Hansen, Anne Auger, Raymond Ros, Steffen Finck, and Petr Pošík. Comparing results of 31 algorithms from the black-box optimization benchmarking BBOB-2009. In *Proceedings of the 12th annual conference companion on Genetic and evolutionary computation (GECCO-2010)*, pages 1689–1696. ACM, 2010.
- [3] A. Auger and N. Hansen. A restart CMA evolution strategy with increasing population size. In *Proceedings of The IEEE Congress on Evolutionary Computation (CEC-2005)*, volume 2, pages 1769–1776. IEEE, 2005.
- [4] Ilya Loshchilov and Frank Hutter. CMA-ES for hyperparameter optimization of deep neural networks. *CoRR*, abs/1604.07269, 2016.
- [5] Kento Uchida, Ryoki Hamano, Masahiro Nomura, Shota Saito, and Shinichi Shirakawa. CMA-ES for discrete and mixed-variable optimization on sets of points. In *International Conference on Parallel Problem Solving from Nature*, pages 236–251. Springer, 2024.
- [6] Ryoki Hamano, Shota Saito, Masahiro Nomura, and Shinichi Shirakawa. CMA-ES with margin: Lower-bounding marginal probability for mixed-integer black-box optimization. In *Proceedings of the genetic and evolutionary computation conference*, pages 639–647, 2022.
- [7] Masahiro Nomura and Masashi Shibata. `cmaes`: A simple yet practical python library for `cmaes`. *CoRR*, abs/2402.01373, 2024.
- [8] N. Hansen, A. Auger, S. Finck, and R. Raymond. Real-Parameter Black-Box Optimization Benchmarking 2009: Experimental Setup. Technical Report RR-6828, INRIA, 2009.
- [9] Anne Auger and Nikolaus Hansen. Tutorial CMA-ES: evolution strategies and covariance matrix adaptation. In *Proceedings of the 14th Annual Conference Companion on Genetic and Evolutionary Computation*, pages 827–848, 2012.

¹<https://github.com/CyberAgentAILab/cmaes>

²<https://github.com/CMA-ES/pycma>

³<https://github.com/facebookresearch/nevergrad>